

Risk Mitigation in CI/CD Pipelines for Safety-Critical Embedded Systems Using Automated Testing and Rollback Strategies

Mock submission draft for review and rewriting

Student name: [Insert your name]

Student number: [Insert your student number]

Campus: [Insert campus name]

Module: ITDMA3-22 Research Design and Methodology

Referencing style: Harvard author-date

Approximate article word count excluding references: 3586

Review note: This is a model draft to rewrite, verify and declare according to your school's AI-use process.

Abstract

Safety-critical embedded systems increasingly need faster software delivery, but continuous integration and continuous deployment cannot be adopted in these contexts as if they were ordinary web application practices. In embedded and cyber-physical systems, a defective deployment may interact with hardware, timing constraints, certification evidence and operational safety. This mock article reviews eight peer-reviewed studies on CI/CD in cyber-physical systems, continuous deployment in systems-of-systems, automated embedded testing, mutation testing, digital twin prototypes, continuous certification and graceful degradation. The analysis shows that deployment risk can be reduced when automated testing is treated as a layered assurance process rather than a single build-server activity. Simulation and digital twin tests improve speed and repeatability, Hardware-in-the-Loop testing improves realism, mutation testing strengthens test adequacy, and continuous certification supports traceability. However, rollback alone is not always sufficient in safety-critical settings because a system may need fail-safe, fail-silent, fail-operational or degraded behaviour.

The article recommends a risk-aware CI/CD pipeline that combines staged deployment, monitoring, assurance evidence and recovery-mode selection.

Keywords

continuous integration; continuous deployment; safety-critical embedded systems; automated testing; rollback; graceful degradation; Hardware-in-the-Loop; digital twin prototypes

1. Introduction

Continuous integration and continuous deployment are attractive because they shorten feedback loops, expose defects earlier and help teams deliver software in smaller increments. In ordinary software products, these advantages can support fast experimentation and frequent release cycles. Safety-critical embedded systems create a more difficult situation. Their software is closely connected to physical devices, sensors, actuators, timing behaviour and regulatory evidence. A deployment fault may not only break a feature; it may cause unsafe system behaviour or weaken confidence in certification evidence. For this reason, the central issue is not whether CI/CD should be used, but how it should be adapted so that speed does not undermine assurance.

Research on CI/CD in cyber-physical systems confirms that these systems face constraints that are less visible in web-based deployment environments. Zampetti et al. (2022) found that cyber-physical systems require a balance between continuous and periodic builds, careful use of simulators and Hardware-in-the-Loop (HIL), and collaboration between hardware and software specialists. Dakkak et al. (2023) similarly show that continuous deployment in software-intensive systems-of-systems requires orchestrated validation, monitoring, documentation and management support. These studies suggest that risk mitigation must be built into the pipeline itself rather than treated as an activity that happens after deployment.

This topic is significant for software engineering because embedded products are increasingly software-defined. Automotive systems, avionics, industrial controllers and medical devices are no longer changed only through rare hardware replacement cycles. Their behaviour can be modified through software updates, which means that

deployment practice becomes part of the safety argument. A poorly controlled pipeline can introduce faults quickly, but a well-designed pipeline can create earlier feedback, stronger evidence and safer recovery options. The research problem is therefore both technical and methodological: engineers need test automation, but they also need a defensible way to decide which evidence is strong enough before deployment.

The aim of this article is to evaluate how automated testing and rollback strategies can reduce deployment risks in safety-critical embedded systems. The objectives are to identify key risks associated with automated deployment, examine automated testing techniques suitable for embedded CI/CD pipelines, and propose a risk-aware pipeline that includes rollback and fail-safe mechanisms. The research question is: How can automated testing and rollback strategies within CI/CD pipelines reduce deployment risks in safety-critical embedded systems?

2. Methodology

This article uses a structured literature review approach based on eight peer-reviewed research papers selected from academic databases and publisher repositories. The search was conducted around terms such as continuous integration, continuous deployment, cyber-physical systems, embedded software testing, safety-critical software, mutation testing, Hardware-in-the-Loop, digital twin prototypes, continuous certification, rollback, fail-safe behaviour and graceful degradation. The selected sources were drawn from ACM, IEEE, Elsevier journals and academic institutional repositories. The review does not claim to be a full systematic literature review, but it follows a structured comparison method so that the selected papers can be analysed consistently.

The inclusion criteria were direct relevance to CI/CD, embedded testing, safety-critical assurance or recovery mechanisms; peer-reviewed publication status; and enough methodological detail to identify research approach, data collection, sampling and analysis. Vendor-only material, blog posts, non-peer-reviewed opinion pieces and general DevOps sources with no embedded or safety-critical relevance were excluded. One older source, Baker and Habli (2013), was included because it provides strong

empirical evidence about mutation testing for safety-critical airborne software and remains directly relevant to test adequacy.

Each paper was analysed using the following comparison fields: article structure, purpose, methodology, research approach, theory or framework, data collection, sampling, data analysis, key findings, findings that confirm prior work, findings that extend existing research, findings that contradict or qualify earlier work, literature gaps and relevance to the research question. This matrix-based method is useful because embedded testing literature is broad and fragmented, a problem also identified in the systematic mapping work of Garousi et al. (2018).

The review also considered the strength and limitations of each type of evidence. Interview and case-study research is valuable for understanding industrial practice, but it may depend heavily on organisational context. Formal modelling provides precise reasoning about failure handling, but it may simplify real operational conditions. Prototype evaluations demonstrate technical feasibility, but they do not always prove generalisability across domains. Systematic mapping gives breadth, but it may be less specific about a single pipeline architecture. By comparing these evidence types together, the article avoids treating all papers as if they make the same kind of claim.

3. Common Sections and Article Structure

The selected papers generally follow the expected structure of research articles, but the emphasis differs according to methodology. The abstract summarises the research context, objective, method and contribution. In empirical papers such as Zampetti et al. (2022) and Dakkak et al. (2023), the abstract is concise and structured around problem, method, findings and conclusion. In design-oriented papers such as Du et al. (2022), the abstract foregrounds the proposed platform and its demonstrated feasibility. The writing style is objective, compressed and evidence-focused.

The introduction establishes the research problem and motivates the study. In CI/CD and safety-critical software papers, the introduction typically contrasts the benefits of faster software delivery with domain constraints such as hardware dependency, certification and field validation. The literature review or background section defines key

concepts and positions the study within prior work. Review papers such as Garousi et al. (2018) devote more space to search strategy and classification, while formal or design-science papers spend more space explaining models, assumptions and technical foundations.

The methodology section explains how evidence was produced. In Zampetti et al. (2022), the method is interview-based and supported by survey validation. In Dakkak et al. (2023), the method is an industrial case study. In Baker and Habli (2013), the method is an empirical evaluation of mutation testing on safety-critical software. In Becker, Voss and Schätz (2018), the method is formal modelling and constraint-based analysis. The results section presents findings or model outcomes, while the discussion interprets what those findings mean in relation to prior research, theory, practice and limitations. The conclusion closes the research question and usually identifies future work.

These structural differences matter because they affect how each article can be used. An empirical study can support claims about real organisational barriers, but it may not prescribe a complete technical architecture. A formal modelling paper can support recovery logic, but it may not show how teams behave in practice. A systematic mapping study can justify the range of testing techniques, but it cannot prove that one technique is always best. Therefore, the structure of each paper helps determine the strength of its evidence and the role it should play in the final synthesis.

4. Literature Review

The first major theme is that CI/CD in cyber-physical and embedded systems requires a different risk model from ordinary software deployment. Zampetti et al. (2022) found that organisations developing cyber-physical systems must balance continuous and periodic builds because not every verification activity can run cheaply or instantly. Simulators are useful for speed, but they may behave differently from physical hardware. HIL improves realism, but it is more expensive, slower and harder to scale. This creates a testing trade-off: early pipeline stages should be fast and repeatable, while later stages should increase environmental realism.

Dakkak et al. (2023) extend this argument into software-intensive systems-of-systems. Their case study of Ericsson 3G Radio Access Network software shows that continuous deployment changes field testing and validation from a late-stage activity into a continuous process. This is important for safety-critical embedded systems because deployment risk is rarely isolated to one component. A change may affect a constituent system, a legacy feature or an interaction between systems. The study therefore supports staged deployment, active monitoring, documentation readiness and orchestration across system boundaries.

The second theme is automated testing for embedded systems. Garousi et al. (2018) show that embedded software testing is a broad field with many techniques, artefacts and application domains. Their systematic mapping supports the idea that no single test technique is enough for embedded assurance. Du et al. (2022) provide a concrete automotive example by proposing a continuous integration multitest platform that combines software engineering CI practices, a commercial virtual platform and FPGA-based verification. This supports the use of layered test environments: virtual tests can accelerate feedback, while hardware-based tests improve confidence in system behaviour.

Test adequacy is also crucial. Baker and Habli (2013) found that mutation testing can reveal weaknesses in safety-critical airborne software even when traditional coverage requirements have already been satisfied. This is significant because CI pipelines often treat test success as a binary pass or fail signal. In safety-critical systems, a passing test suite may still be weak if it does not expose meaningful faults. Mutation testing can therefore act as a stronger assurance gate by evaluating whether the test suite is capable of detecting seeded faults.

The third theme is the use of digital twins and continuous certification. Barbie and Hasselbring (2024) formalise digital twin prototypes and argue that they can support automated testing of embedded software without constant access to the physical object. This does not remove the need for HIL, but it can reduce dependence on scarce hardware and improve repeatability. Baron and Louis (2023) focus on safety-critical avionics and argue that certification requirements should be integrated continuously into

development rather than handled as late audit work. Together, these studies suggest that a risk-aware pipeline should produce both technical test results and traceable assurance artefacts.

The final theme is recovery after faults escape testing. Becker, Voss and Schätz (2018) analyse graceful degradation in fault-tolerant automotive systems and show that safety-critical systems need failure-handling actions such as fail-silent, fail-safe, fail-operational and degraded modes. This matters because rollback is often discussed as if reverting to an older software version always solves deployment risk. In embedded safety-critical systems, the safer response may depend on operational context. A system may need to isolate a failed component, reduce functionality, switch to a redundant deployment or enter a defined safe state.

Across these themes, the literature suggests a layered risk-control logic. Early stages of the pipeline should maximise speed and coverage because developers need fast feedback. Middle stages should increase environmental realism because embedded failures may depend on timing, hardware interaction or physical signals. Later stages should focus on evidence, monitoring and recovery because not every risk can be eliminated before deployment. This progression is important because it prevents two weak extremes: relying only on cheap software tests, or delaying all meaningful validation until late-stage hardware testing.

5. Results and Comparative Analysis

The selected articles differ in structure because they use different methodologies. Zampetti et al. (2022) and Dakkak et al. (2023) are empirical software engineering studies, so their structures emphasise research questions, data collection, findings and validity. Garousi et al. (2018) is a systematic mapping study, so it foregrounds search strategy, inclusion criteria, classification and synthesis. Du et al. (2022) and Baron and Louis (2023) are design-oriented contributions, so they focus on proposed frameworks, tooling and industrial demonstration. Becker, Voss and Schätz (2018) uses a formal method structure, with strong emphasis on system modelling and constraint-based analysis.

The research approaches also differ. Zampetti et al. (2022) uses an interpretivist qualitative core because it relies on semi-structured interviews and open card sorting, but it adds survey validation with 55 professional developers. Dakkak et al. (2023) uses a pragmatic industrial case-study approach. Baker and Habli (2013) is closer to a positivist empirical evaluation because it examines mutation testing results on safety-critical C and Ada software. Garousi et al. (2018) uses positivist evidence synthesis through systematic mapping. Barbie and Hasselbring (2024) and Baron and Louis (2023) are mainly design-science oriented because they develop concepts, frameworks or tooling. Becker, Voss and Schätz (2018) is formal analytical research because it uses a system model and SMT-based constraint solving.

The papers agree that automated testing is necessary but not sufficient on its own. Zampetti et al. (2022), Garousi et al. (2018) and Du et al. (2022) all support layered testing, although they approach it differently. Zampetti et al. (2022) emphasise organisational practice and CPS barriers, Garousi et al. (2018) map the wider testing landscape, and Du et al. (2022) demonstrate a practical automotive CI platform. These findings confirm prior software engineering assumptions that early testing reduces risk, but they also extend them by showing that embedded testing requires hardware-aware validation.

The data collection and sampling strategies also reveal important limitations. Zampetti et al. (2022) collected interview and survey evidence from professionals, which is strong for identifying practice-based barriers but depends on participant interpretation. Dakkak et al. (2023) provides deep industrial evidence from one telecommunications case, which improves realism but limits direct generalisation. Du et al. (2022) evaluates a concrete automotive platform, which shows technical feasibility but not necessarily organisational adoption. Baker and Habli (2013) uses safety-critical airborne software, giving strong relevance to assurance, but the study is not specifically about CI/CD. These limitations do not weaken the review; rather, they show why synthesis is necessary.

Several papers extend existing research by moving beyond simple pipeline automation. Dakkak et al. (2023) show that continuous deployment in systems-of-

systems requires field validation orchestration rather than isolated component release. Baron and Louis (2023) extend Agile and CI thinking into certification by proposing continuous certification in safety-critical avionics. Barbie and Hasselbring (2024) extend simulation-based testing by formalising digital twin prototypes as CI/CD-friendly test artefacts. Becker, Voss and Schätz (2018) extend recovery thinking by showing that system behaviour after failure should be modelled through redundancy and degradation scenarios.

The main contradiction or qualification across the literature concerns the transferability of mainstream CI/CD. The reviewed papers do not reject CI/CD, but they contradict the assumption that speed alone is the goal. In web software, rollback may be acceptable when a release causes a defect. In safety-critical embedded systems, rollback must be evaluated against operational safety, hardware state, certification evidence and system criticality. A rollback that restores an older version may still be unsafe if the system has already entered a hazardous state or if the older version is incompatible with current hardware configuration. Therefore, rollback should be part of a wider fail-safe and degradation strategy rather than the only recovery mechanism.

The reviewed findings therefore influence the proposed research direction in a practical way. A future research article should not merely ask whether CI/CD is possible in safety-critical embedded systems. The stronger question is which pipeline controls are needed at each risk level. Low-risk code changes may be handled through automated software tests and staged rollout. Medium-risk changes may require simulation, digital twin validation and expanded monitoring. High-risk changes may require HIL, mutation adequacy checks, traceability evidence and explicit safety sign-off. This risk-tiered view connects the literature to a realistic engineering process.

6. Discussion

The findings answer the research question by showing that automated testing and rollback reduce deployment risk at different points in the pipeline. Automated testing reduces pre-deployment risk by detecting defects earlier, increasing repeatability and producing evidence about software behaviour. Rollback and fail-safe strategies reduce post-deployment risk by limiting the consequences of faults that escape testing. In

safety-critical embedded systems, these two ideas must be connected. A pipeline that tests well but has no recovery plan is incomplete, while a pipeline that can rollback but lacks strong test evidence is reactive rather than risk-aware.

A risk-aware CI/CD pipeline should therefore be layered. The first stage should include build checks, static analysis, unit tests and requirements traceability checks. The second stage should use simulation or digital twin prototypes for fast integration tests, supported by the digital twin argument of Barbie and Hasselbring (2024). The third stage should use HIL or FPGA-supported validation for tests where timing, physical interaction or hardware behaviour matters, as suggested by Zampetti et al. (2022) and Du et al. (2022). The fourth stage should include safety assurance checks such as mutation testing, structural coverage review and certification artefact generation, drawing on Baker and Habli (2013) and Baron and Louis (2023). The fifth stage should use staged deployment, field monitoring and validation, following Dakkak et al. (2023). The final stage should define rollback, fail-safe and graceful degradation rules before deployment, supported by Becker, Voss and Schätz (2018).

This model also shows why methodology matters. The interview studies identify barriers that engineers experience in practice. The case study shows how deployment and field validation change in a real organisation. The prototype study shows how CI can incorporate virtual and hardware-supported testing. The mutation-testing study strengthens the quality argument. The formal degradation study explains recovery logic. No single paper provides the whole answer, but together they form a coherent argument: deployment risk is reduced when the pipeline combines organisational readiness, technical testing, assurance evidence and runtime recovery planning.

The main literature gap is synthesis. The reviewed studies cover important parts of the problem, but few integrate all parts into one end-to-end CI/CD model for safety-critical embedded systems. CI/CD studies often emphasise organisational challenges and test environments. Testing studies often focus on techniques such as mutation testing, HIL or digital twins. Certification studies focus on evidence and compliance. Fault-tolerance studies focus on recovery behaviour. A distinction-level research

contribution is therefore to connect these areas and argue that deployment risk must be managed across the full lifecycle.

Another gap is contextual. Strong evidence exists in avionics, automotive, telecommunications and cyber-physical systems, but the findings may not transfer equally across all safety-critical domains. For example, avionics certification has different regulatory constraints from automotive software or medical devices. There is also a measurement gap: the literature does not yet provide a single standard metric for deciding when a pipeline has reduced deployment risk enough for a specific safety-critical release. This suggests that future research should develop evaluation metrics that combine test adequacy, hardware realism, certification evidence, monitoring coverage and recovery readiness.

From an evaluation perspective, the proposed pipeline should be judged by both prevention and response. Prevention asks whether the pipeline catches faults early, exercises realistic conditions and maintains traceability. Response asks whether the system can detect a bad deployment, limit exposure and enter a safe or degraded state. This distinction is important because safety-critical risk cannot be reduced to defect count alone. A system with fewer defects but poor recovery behaviour may still be dangerous, while a system with strong recovery but weak testing may expose users to avoidable failures.

7. Recommendations

The first recommendation is that safety-critical embedded CI/CD should use progressive assurance gates rather than a single automated test stage. Fast software-only tests should run early, but physical or high-fidelity validation should be reserved for changes that create hardware, timing or safety risk. The second recommendation is to treat mutation testing and coverage evidence as quality signals, not as administrative extras. A passing test suite should not be assumed adequate unless its fault-detection strength has been evaluated.

The third recommendation is to include certification artefacts in the pipeline. Requirements traceability, test evidence, review records and configuration information

should be generated continuously so that safety assurance does not become a late-stage documentation exercise. The fourth recommendation is to define rollback and fail-safe behaviour before deployment. For low-risk failures, rollback may be enough. For safety-critical failures, the system may need graceful degradation, redundancy, component isolation or a safe shutdown mode. Finally, staged deployment and monitoring should be used to detect field deviations before a release reaches the full operational population.

8. Proposed Conference Venue

A suitable conference for this article would be the International Conference on Software Testing, Verification and Validation (ICST). ICST is appropriate because its scope includes software testing theory and practice, verification and validation, empirical studies, tools, embedded software and technology transfer (ICST Steering Committee, 2023). The article's focus on automated testing, HIL, mutation testing, digital twin test environments and deployment risk in embedded systems fits this venue better than a general software engineering conference. If developed further, the article could be positioned as a literature-based conceptual model for risk-aware CI/CD in safety-critical embedded systems.

9. Conclusion

Automated testing and rollback strategies can reduce deployment risks in safety-critical embedded systems when they are implemented as part of a broader risk-aware CI/CD pipeline. The literature shows that embedded systems require more than fast builds and frequent releases. They require hardware-aware testing, test adequacy evidence, continuous certification artefacts, staged deployment, monitoring and predefined recovery behaviour. Rollback is useful, but it should not be treated as the only safety mechanism. In many safety-critical contexts, fail-safe behaviour, fail-operational redundancy or graceful degradation may be more appropriate than simple version reversal. The central conclusion is that CI/CD can support safety-critical embedded development only when it is adapted to the assurance, hardware and recovery demands of the domain.

References

- Zampetti, F., Tamburri, D.A., Panichella, S., Panichella, A., Di Penta, M. and Canfora, G. (2022) 'Continuous integration and delivery practices for cyber-physical systems: An interview-based study', *ACM Transactions on Software Engineering and Methodology*, 32(3), Article 73. Available at: <https://doi.org/10.1145/3571854>.
- Dakkak, A., Bosch, J., Holmström Olsson, H. and Mattos, D.I. (2023) 'Continuous deployment in software-intensive system-of-systems', *Information and Software Technology*, 159, Article 107200. Available at: <https://doi.org/10.1016/j.infsof.2023.107200>.
- Du, B., Azimi, S., Moramarco, A., Sabena, D., Parisi, F. and Sterpone, L. (2022) 'An automated continuous integration multitest platform for automotive systems', *IEEE Systems Journal*, 16(2), pp. 2495-2506. Available at: <https://doi.org/10.1109/JSYST.2021.3069548>.
- Baker, R. and Habli, I. (2013) 'An empirical evaluation of mutation testing for improving the test quality of safety-critical software', *IEEE Transactions on Software Engineering*, 39(6), pp. 787-805. Available at: <https://doi.org/10.1109/TSE.2012.56>.
- Garousi, V., Felderer, M., Karapıçak, Ç.M. and Yılmaz, U. (2018) 'Testing embedded software: A survey of the literature', *Information and Software Technology*, 104, pp. 14-45. Available at: <https://doi.org/10.1016/j.infsof.2018.06.016>.
- Barbie, A. and Hasselbring, W. (2024) 'From digital twins to digital twin prototypes: Concepts, formalization, and applications', *IEEE Access*, 12, pp. 75337-75365. Available at: <https://doi.org/10.1109/ACCESS.2024.3406510>.
- Baron, C. and Louis, V. (2023) 'Framework and tooling proposals for Agile certification of safety-critical embedded software in avionic systems', *Computers in Industry*, 148, Article 103887. Available at: <https://doi.org/10.1016/j.compind.2023.103887>.
- Becker, K., Voss, S. and Schätz, B. (2018) 'Formal analysis of feature degradation in fault-tolerant automotive systems', *Science of Computer Programming*, 154, pp. 89-133. Available at: <https://doi.org/10.1016/j.scico.2017.10.007>.

ICST Steering Committee (2023) 'The International Conference on Software Testing, Verification and Validation - ICST'. Available at: <https://icstconference.github.io/> (Accessed: 4 June 2026).

AI Assistance Declaration Draft

Draft wording for the separate AI declaration form:

I used OpenAI Codex as an academic support tool to help plan the assignment, identify relevant peer-reviewed sources, create a literature matrix, structure the article, generate a mock draft for review, and check Harvard-style citation placement. I reviewed, edited and rewrote the final submission in my own words, verified the sources, and take responsibility for the submitted content.